Avaliação Arquitetural do Projeto WebMarket

DDD, microsserviços, Java, Quarkus, eventos, cache e consistência distribuída

Resumo executivo

A proposta é adequada como prova de conceito de DDD, microsserviços e mensageria. Entretanto, o desenho original atribui ao cache uma responsabilidade de negócio que ele não deveria possuir: decidir ou garantir a disponibilidade de estoque.

Recomendação central: o Serviço Produto deve ser o único proprietário do estoque e das reservas. O banco do contexto de Produto é a fonte de verdade; o cache é apenas uma projeção de leitura. A confirmação do pedido deve decorrer de uma reserva efetiva de estoque, e não de uma consulta anterior ao cache.

1. Problema principal do fluxo original

```
CriarPedido
-> consultar estoque no cache
-> criar Pedido
-> emitir PedidoCriadoEvent
-> reduzir estoque no cache
-> ConfirmarPedido
-> verificar novamente e reduzir estoque
```

- Não está claro quem é o proprietário da verdade sobre o estoque.
- A redução pode ocorrer em duas etapas diferentes.
- Uma consulta ao cache não garante que a quantidade continuará disponível.
- Dois pedidos concorrentes podem ler o mesmo saldo e vender mais unidades do que existem.

2. Bounded contexts sugeridos

```
br.gov.caixa.webmarket

├── catalogo/produto
│   ├── Produto
│   ├── Estoque
│   └── ReservaEstoque
├── pedido
│   ├── Pedido
│   ├── ItemPedido
│   ├── EnderecoEntrega
│   └── StatusPedido
└── cliente
    └── Cliente
```

A separação física pode resultar em webmarket-carga, webmarket-produto e webmarket-pedido. Porém, microsserviço não é automaticamente um bounded context: a divisão deve acompanhar a propriedade dos dados, o vocabulário e as regras de negócio.

3. Serviço Carga

O serviço pode existir para gerar massa de testes e inicializar a PoC, mas não deve inserir diretamente nos bancos dos demais serviços nem ser proprietário do cache de estoque.

- Criar produtos por meio da API do Serviço Produto.
- Criar clientes por meio da API do contexto responsável.
- Gerar dados aleatórios somente como função de apoio.
- Deixar a manutenção do cache sob responsabilidade do Serviço Produto.

4. Serviço Produto e reserva de estoque

Os endpoints ConsultarProdutoQtdEstoque e ReduzirEstoque não representam completamente o ciclo de negócio. O conceito de reserva evita que uma simples leitura seja confundida com garantia de compra.

```
ConsultarProduto
ConsultarDisponibilidade
ReservarEstoque
ConfirmarReserva
LiberarReserva
```

Exemplo de disponibilidade: estoqueDisponivel = estoqueFisico - estoqueReservado. A operação ReservarEstoque deve ser atômica e tratar concorrência no banco do Serviço Produto.

5. Commands e Events

Commands - pedido para executar algo	Events - fato ocorrido
CriarPedido ReservarEstoque ConfirmarReserva CancelarPedido LiberarEstoque	PedidoCriadoEvent EstoqueReservadoEvent ReservaEstoqueRejeitadaEvent PedidoConfirmadoEvent PedidoCanceladoEvent

O retorno HTTP de POST /pedidos deve ser um DTO com pedidoId, valor e status. O PedidoCriadoEvent deve ser publicado para comunicar internamente que o pedido já foi criado; ele não deve ser usado como resposta síncrona da API.

6. Fluxo recomendado

```
Cliente -> POST /pedidos -> Pedido Service
Pedido Service: cria Pedido com status CRIADO
Pedido Service -> PedidoCriadoEvent -> Broker
Produto Service: tenta reservar estoque
sucesso -> EstoqueReservadoEvent
falha -> ReservaEstoqueRejeitadaEvent
Pedido Service:
EstoqueReservadoEvent -> CONFIRMADO
Reserva rejeitada -> CANCELADO
```

Esse fluxo forma uma Saga baseada em eventos. Cada serviço executa sua transação local; quando uma etapa falha, uma ação compensatória pode liberar a reserva ou cancelar o pedido.

7. Papel correto do cache

```
Produto Service
├── Banco: fonte de verdade
└── Cache: projeção para consulta rápida
```

Uma resposta de cache informando cinco unidades significa que a última visão disponível indica esse saldo. Ela não garante cinco unidades ao comprador. A garantia nasce somente após ReservarEstoque ser concluído no agregado responsável.

8. Transactional Outbox

Salvar o pedido e depois publicar o evento produz uma janela de inconsistência: o commit do banco pode funcionar e a publicação no broker pode falhar. O padrão Transactional Outbox registra o dado de negócio e a mensagem de saída na mesma transação local.

```
TRANSAÇÃO LOCAL
INSERT/UPDATE PEDIDO
INSERT OUTBOX_EVENT (PedidoCriadoEvent)
COMMIT

Publicador/CDC -> envia o evento ao broker
```

9. Idempotência

Mensagens podem ser entregues novamente. O processamento de uma reserva precisa reconhecer que pedidoId + produtoId já foi tratado, evitando nova redução do estoque.

```
chave lógica: pedidoId + produtoId
primeira entrega -> reserva
entrega duplicada -> retorna o resultado anterior / ignora com segurança
```

10. Estados do agregado Pedido

```
CRIADO -> AGUARDANDO_ESTOQUE -> CONFIRMADO
└----> CANCELADO (ESTOQUE_INSUFICIENTE)
```

As transições devem estar encapsuladas no domínio, por exemplo pedido.confirmar() e pedido.cancelar(motivo), em vez de alterar o status diretamente por setters.

11. Estrutura sugerida no Quarkus

```
br.gov.caixa.webmarket.pedido
├── domain
│   ├── model
│   ├── event
```

```
│ └── repository
├── application
│ ├── CriarPedidoUseCase
│ ├── ConfirmarPedidoUseCase
│ └── CancelarPedidoUseCase
├── infrastructure
│ ├── persistence
│ ├── messaging
│ └── cache
└── presentation
└── PedidoResource
```

12. Comparativo final

Desenho original	Recomendação
Cache determina disponibilidade	Banco/agregado determina; cache apenas consulta
ReduzirEstoque durante criação	ReservarEstoque e depois confirmar/liberar
Evento usado como resposta	HTTP retorna DTO; evento segue para mensageria
Confirmar verifica e reduz novamente	Confirmar uma reserva já realizada
Mensagem “Impossível comprar”	ReservaEstoqueRejeitadaEvent com motivo
Carga controla cache	Produto controla sua projeção/cache
DB + publicação direta	Transactional Outbox
Consumidor simples	Consumidor idempotente

Conclusão

A arquitetura fica mais coerente com DDD quando Pedido e Estoque são agregados dos seus próprios contextos, o cache é tratado como modelo de leitura e a confirmação decorre da reserva efetiva. Para robustez distribuída, o desenho deve prever Saga, consistência eventual, Transactional Outbox e consumidores idempotentes.

Referências técnicas

- Transactional Outbox: microservices.io/patterns/data/transactional-outbox.html
- Debezium Quarkus Outbox: quarkus.io/extensions/io.debezium/debezium-quarkus-outbox-reactive/